

Apunte teórico: Integración de Python con Bases de Datos Relacionales (MySQL)

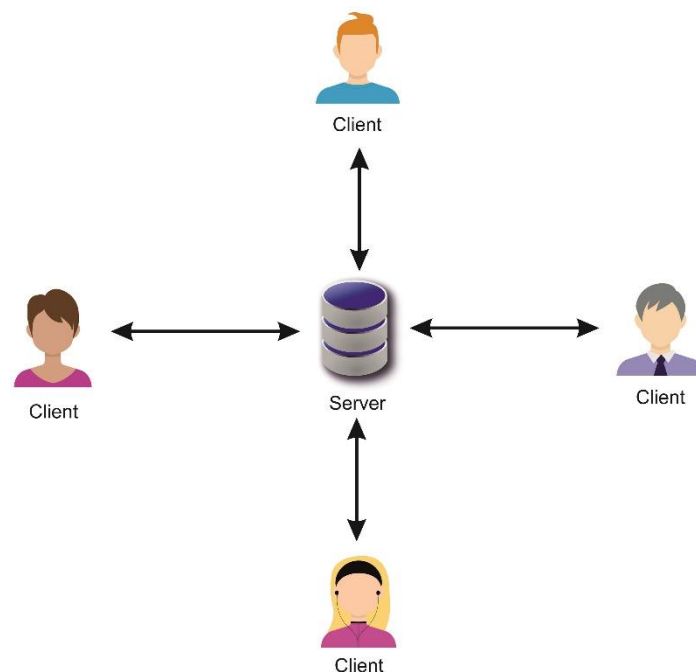
El presente documento aborda el salto definitivo hacia los entornos de producción reales. Hasta esta etapa, nuestro análisis se basó en la ingesta de archivos planos (CSV, Excel). A partir del Módulo 5, el objetivo principal es conectar el análisis con sistemas de persistencia relacional. Aprenderemos a establecer comunicaciones seguras entre nuestros scripts y un motor de base de datos, extrayendo información dinámica en tiempo real para la toma de decisiones.

1. El Límite del Archivo Plano y la Arquitectura Cliente-Servidor

Trabajar con archivos .csv es excelente para el aprendizaje inicial y el análisis exploratorio estático. Sin embargo, en el ámbito corporativo, los archivos locales presentan fallas críticas:

- **Desactualización:** Un archivo descargado queda obsoleto un segundo después de guardarse.
- **Concurrencia:** Varios analistas no pueden escribir o leer el mismo archivo al mismo tiempo sin corromperlo.
- **Seguridad:** Un archivo local carece de control de accesos, cifrado y auditoría.
- **Rendimiento:** Cargar un CSV de 10 millones de filas satura la memoria RAM (Memoria Principal) de cualquier computadora estándar.

Para solucionar esto, la industria del software utiliza la **Arquitectura Cliente-Servidor**.



En este modelo, las responsabilidades se dividen:

- **El Servidor (MySQL):** Es un software robusto dedicado exclusivamente a almacenar, indexar y proteger la información. Espera peticiones y cuenta con un motor optimizado para realizar cálculos (agrupaciones, sumas) sobre millones de registros en milisegundos.
- **El Cliente (Nuestros scripts en Python):** Es la aplicación que solicita información. Se conecta al servidor, envía una petición (Consulta SQL), recibe los resultados precisos y los formatea para su visualización.

2. Los Traductores: Instalación de conectores (pymysql, sqlalchemy)

Python y MySQL hablan idiomas diferentes. Para que se entiendan, necesitamos intermediarios de software conocidos como **conectores** o **drivers**.

1. **PyMySQL (El Driver):** Es la librería de bajo nivel que sabe cómo comunicarse con el protocolo de red de MySQL. Es el "cable" físico y lógico que une nuestro código con el puerto del servidor.
2. **SQLAlchemy (El Motor/ORM):** Es el estándar de la industria en Python para el manejo de bases de datos. Actúa como un administrador de la conexión (Connection Pool). Pandas requiere estrictamente SQLAlchemy para ejecutar sus métodos de lectura y escritura de forma segura y eficiente.

Instalación en el entorno de desarrollo

pip install pymysql sqlalchemy

3. Connection Strings y seguridad de credenciales

Para que el servidor MySQL nos deje entrar a leer los datos, debemos presentarle un "pasaporte" con nuestras credenciales de acceso. Esta llave maestra se denomina **Cadena de Conexión (Connection String)**.

Una cadena de conexión es una URL que concentra todos los parámetros de red y seguridad. Su anatomía estándar utilizando SQLAlchemy es la siguiente:

dialecto+driver://usuario:contraseña@host:puerto/base_de_datos

Desglosando los componentes:

- **Dialecto+Driver (mysql+pymysql):** Le indica a SQLAlchemy con qué tipo de base de datos hablará y qué traductor usará.
- **Usuario y Contraseña (root:password):** Las credenciales del analista o de la aplicación.
- **Host (localhost o IP):** La dirección física o virtual donde vive el servidor. En etapa de desarrollo suele ser localhost (la propia máquina).
- **Puerto (3306):** La "puerta" de red específica por la que escucha MySQL.
- **Base de Datos (logistica_db):** El esquema exacto dentro del servidor sobre el cual vamos a operar.

Importante (Seguridad de Credenciales): En un entorno local educativo, es común escribir las contraseñas en texto plano. Sin embargo, en el ámbito profesional, incrustar contraseñas en el código fuente (Hardcoding) es una vulnerabilidad crítica. En producción, las contraseñas deben inyectarse mediante Variables de Entorno (.env) o gestores de secretos.

4. Ejecución de queries SQL desde Python y volcado a DataFrames

Una vez establecido el puente (el engine de SQLAlchemy), el ciclo de trabajo se reduce a un solo paso magistral gracias a Pandas.

Ya no es necesario instanciar "cursores" de bases de datos, ni iterar sobre los resultados fila por fila mediante bucles *for* nativos de Python. Pandas provee el método *pd.read_sql_query()*, diseñado específicamente para la ejecución de queries SQL desde Python y volcado a DataFrames de manera instantánea.

Este método recibe dos parámetros fundamentales:

1. **La Query:** Una consulta SQL válida escrita en formato de texto (String).
2. **La Conexión (con):** El motor instanciado por SQLAlchemy.

```
import pandas as pd
```

```
from sqlalchemy import create_engine
```

1. Instanciamos el motor

```
engine = create_engine("mysql+pymysql://root:@localhost:3306/logistica_cuyo_db")
```

2. Definimos la consulta

```
consulta_sql = "SELECT Barrio, Valor_ARS FROM envios WHERE Estado_Entrega = 'Completado'"
```

3. Pandas ejecuta la query, mapea los tipos de datos y construye el DataFrame

```
df = pd.read_sql_query(consulta_sql, con=engine)
```

Al utilizar este método, Pandas asume la responsabilidad de mapear los tipos de datos de MySQL (como INT, VARCHAR o DATETIME) a sus equivalentes vectoriales en NumPy y Pandas (int64, object, datetime64), garantizando que la estructura tabular resultante esté inmediatamente lista para las fases de limpieza, agregación matemática y visualización.

5. Arquitectura de Software: ¿Quién debe hacer el esfuerzo computacional?

La integración de SQL y Python nos plantea un dilema de diseño de software fundamental: Si necesitamos agrupar datos (ej. un total de ventas por provincia), ¿hacemos un *GROUP BY* en la consulta SQL, o hacemos un *SELECT ** y luego aplicamos *.groupby()* en Pandas?

La **regla de oro corporativa** establece que el trabajo pesado de agregación, filtrado numérico y cruce de tablas (JOINS) masivos debe delegarse al **Motor de Base de Datos (MySQL)**.

Los motores relacionales están indexados y optimizados a nivel de hardware para realizar estas matemáticas sobre el disco. Extraer millones de registros crudos a través de la red solo para agruparlos en Python saturará el ancho de banda y la memoria RAM del servidor de análisis. Pandas debe recibir, en la medida de lo posible, los resúmenes estadísticos (datasets ya reducidos) para aplicarles lógicas de negocio finas y generar las visualizaciones finales.